IMPLEMENTATION REFERENCE

Exact table optimization

Model, algorithm, optimality, and limitations

Kahanamoku-Meyer and Yao: Appendix F.2 points
Existing common-child-width PhaseProduct planner

16 October 2026

Purpose. Document the existing exhaustive exact algorithm for implementation and practicality tests. This document proposes no search improvement.

Guarantee. A completed, correct execution returns the minimum recursive T-count in the precisely defined model, a recursion sequence, and one realizing table per used arity.

Limit. The established time bound is

$$2^{O(K^4 \log K)} \operatorname{poly}(n).$$

This is a theoretical upper bound, not a completion-time estimate.

Each of the twelve limitations appears in three forms: ordinary language, controlled technical English, and a formal mathematical statement. The model, search procedure, optimality argument and complexity accounting precede the itemization.

Provenance. KMY notation and point selection come from arXiv:2403.18006v4 [1]. The scanner, asymmetric charges, rounded T-count objective, shared-table restriction and exhaustive optimization theorems belong to the local research model [4, 5, 6]. They are not attributed to KMY.

Verification status. The full optimizer is not machine formalized, and no large-arity execution is reported here.

Standalone reference. Inputs, acceptance conditions, costs and stopping claims are defined here. References identify provenance and the full supporting proofs.

Contents

1 Notation and exact model **1**
1.1 Relation to KMY notation . . . 1
1.2 Fixed points and literal target rows . . . 1
1.3 Legal instruction tables . . . 2
1.4 Historical width scanner . . . 2
1.5 Exact recursive objective . . . 3

2 How the exhaustive algorithm finds tables **4**
2.1 Minimum charge under a width cap . . . 4
2.2 Why the search is finite . . . 4
2.3 Normalized macro enumeration . . . 5
2.4 Construct the minimum-charge catalogue . . . 6

3 Why the answer is optimal **6**
3.1 Local optimality . . . 6
3.2 Global recursive optimization . . . 7
3.3 Feasibility certificates and optimality certificates . . . 8
3.4 Is verifying optimality efficient? . . . 8
3.5 A complete small example . . . 9

4 Computational complexity and storage **9**
4.1 Local search . . . 9
4.2 Outer search and full bound . . . 9
4.3 Time and memory interpretation . . . 10

5 Twelve limitations in three variants **11**
5.1 L1. Global optimality has a fixed search domain . . . 11
5.2 L2. The recursion model is restricted . . . 11
5.3 L3. The objective is one specified T-count . . . 12
5.4 L4. The arity cutoff limits the guarantee . . . 12
5.5 L5. The time bound does not establish practicality . . . 13
5.6 L6. The outer planner can remain costly . . . 13
5.7 L7. Minimum local charge is not the recursive objective . . . 14
5.8 L8. Sufficient finite bounds can be large . . . 14
5.9 L9. Exact integers and memory constrain implementation . . . 15
5.10 L10. Faithful semantics are necessary . . . 15
5.11 L11. A partial search does not certify a minimum . . . 16
5.12 L12. The full implementation is not certified . . . 16

6 Implementation result records **17**
6.1 Controlled-English technical vocabulary . . . 18

References and proof provenance **19**

1 Notation and exact model

1.1 Relation to KMY notation

KMY use n for operand bitwidth, k for the number of chunks, $q = 2k-1$ for PhaseProduct evaluations, and w_ℓ for evaluation points. Their operation is [1, Eq. (10)]

$$\mathsf{PhaseProduct}(\phi)|x\rangle|z\rangle = e^{i\phi xz}|x\rangle|z\rangle.$$

Their coefficient vectors list high-degree coefficients first: $\mathbf{x} = (x_{k-1}, \ldots, x_0)^\mathsf{T}$, and similarly $\mathbf{z}$. Their integer matrices A' and A'' give the Appendix B form

$$\phi xz = \sum_{\ell=0}^{q-1} \phi_\ell (A'\mathbf{x})_\ell (A'\mathbf{z})_\ell, \qquad \boldsymbol{\phi}^\mathsf{T} = \phi\, \mathbf{e}_{b_{\mathrm{rad}}}^\mathsf{T} (A'')^{-1}. \tag{KMY–PP}$$

Here $\mathbf{e}_{b_{\mathrm{rad}}}$ has length q. At current width m, our unequal-chunk convention uses $b_{\mathrm{rad}} = 2^{\lfloor m/k \rfloor}$. The identity specifies the required linear forms; the table optimizer does not search over the phase coefficients.

Symbol	Meaning
$n \geq 1,\ K \geq 2$	Root operand bitwidth and maximum allowed arity.
m	Current operand bitwidth; never an evaluation point.
$k,\ q = 2k-1$	Arity and number of required PhaseProduct markers.
w_ℓ, A', A''	KMY evaluation point and integer matrices.
P, X	Complete instruction table and current integer row matrix.
$\gamma(P)$	Positive additions plus three times negative additions plus negations.
(α, β)	Canonical two-parameter historical-width profile.
$W_P(m)$	Common child width specified by the scanner.
C, δ, Λ	Charge budget, profile difference and width cutoff.
N_i, R	Call multiplicity and terminal rotation count $N_D m_D^2$.
$\mathcal{T}, U$	Recursive T-count and the cost of stopping at the root.

The program indexes coefficients in the opposite order. Let J reverse coordinates and put $\mathbf{x}_\uparrow = J\mathbf{x} = (x_0, \ldots, x_{k-1})^\mathsf{T}$. Program row $X_{i,*}$ denotes $X_{i,*}\mathbf{x}_\uparrow$. Its target is the corresponding row of $A'J$. This reversal changes notation, not physical operations.

1.2 Fixed points and literal target rows

The points are the first q entries of the F.2 sequence

$$0,\ \infty,\ -1,\ 1,\ -\tfrac{1}{2},\ \tfrac{1}{2},\ -2,\ 2,\ -\tfrac{1}{4},\ \tfrac{1}{4},\ -4,\ 4, \ldots. \tag{1}$$

For successive $\omega = 2, 4, 8, \ldots$, the next block is $(-\omega^{-1}, \omega^{-1}, -\omega, \omega)$. This selects a set, not a mandatory marker order. KMY Appendix G examples can use different point sets [1, App. F.2 and G].

For coefficient index $j = 0, \ldots, k-1$, define

$$\begin{aligned} [v_k(a)]_j &= a^j && (a \text{ an integer}), \\ [v_k(1/c)]_j &= c^{k-1-j} && (c \in \{\pm 2, \pm 4, \ldots\}), \\ v_k(\infty) &= e_{k-1}. \end{aligned} \tag{2}$$

Use $0^0 = 1$. A negative reciprocal keeps its negative denominator c. Thus $v_4(-1/2) = (-8, 4, -2, 1)$, not $(8, -4, 2, -1)$. Sign equivalence is not literal equality. Every target is primitive: its entries have no common factor 2.

For $k = 3$, the five required rows are

w_ℓ	0	∞	-1	1	$-1/2$
$v_3(w_\ell)$	$(1,0,0)$	$(0,0,1)$	$(1,-1,1)$	$(1,1,1)$	$(4,-2,1)$.

Arbitrary evaluation points are not inputs to this problem.

For full matrix definitions, let J_t reverse t coordinates. The rows in descending order are

$$A'_{\ell,*} = v_k(w_\ell)J_k, \qquad A''_{\ell,*} = v_q(w_\ell)J_q, \qquad \mathbf{e}_{b_{\mathrm{rad}}} = (b_{\mathrm{rad}}^{q-1}, \ldots, b_{\mathrm{rad}}, 1)^{\mathsf{T}}.$$

The rule (2) defines v_q by replacing its vector length k with q, while keeping the same selected points w_ℓ. For infinity this selects the leading coefficient. The distinct points make the scaled interpolation matrix A'' invertible. No numerical interpolation approximation is needed to define the model.

1.3 Legal instruction tables

Start with $X = I_k$. All k rows are writable. For distinct d, h, sign $\varepsilon = \pm 1$, and integer $s \geq 0$:

$$\begin{array}{lll} \mathrm{add}(d,h,\varepsilon,s): & X_d \leftarrow X_d + \varepsilon 2^s X_h, & \text{charge 1 or 3;} \\ \mathrm{left}(d,s): & X_d \leftarrow 2^s X_d, & \text{charge 0;} \\ \mathrm{right}(d,s): & X_d \leftarrow X_d/2^s, & \text{charge 0;} \\ \mathrm{neg}(d): & X_d \leftarrow -X_d, & \text{charge 1;} \\ \mathrm{mark}(d,\ell): & \text{require } X_d = v_k(w_\ell), & \text{charge 0.} \end{array} \tag{3}$$

An addition costs 1 when $\varepsilon = 1$ and 3 when $\varepsilon = -1$. A right shift is legal only when every entry of the selected row is divisible by 2^s. A marker changes no row.

Acceptance requires every target exactly once and final matrix exactly I_k, with the original row indices and signs. Every intermediate row must be integral. Operations are invertible over $\mathbb{Q}$, so no reachable row is zero.

Arbitrary legal helper rows, helper sharing and marker permutations are allowed. There is no separate reset, helper-shape, determinant-size or instruction-count restriction. A finite minimum-witness bound is a theorem about this universe, not an extra constraint on it.

1.4 Historical width scanner

At operand width $m = kL + r$, $0 \leq r < k$, initial chunk widths are $(L, \ldots, L, L + r)$. The scanner updates are

$$\begin{array}{ll} \mathrm{add}(d,h,\varepsilon,s): & u_d \leftarrow 1 + \max(u_d, u_h + s), \\ \mathrm{left}(d,s): & u_d \leftarrow u_d + s, \\ \mathrm{right}(d,s): & u_d \leftarrow \max(0, u_d - s), \\ \mathrm{neg}(d): & u_d \leftarrow u_d + 1. \end{array} \tag{4}$$

Markers change nothing. Let $H(P; u_{\mathrm{in}})$ be the maximum of all coordinates at all boundaries, including the initial boundary. Define

$$W_P(m) = 1 + H(P; (L, \ldots, L, L + r)). \tag{5}$$

This conservative specified scanner is not an exact signed-value range analysis or a branch-specific KMY width calculation.

Put $R_k = k - 1$. Two complete scans determine the width function:

$$\begin{aligned} H_0 &= H(P; (0, \dots, 0)), \\ H_R &= H(P; (0, \dots, 0, R_k)), \\ \alpha(P) &= H_0, \qquad \beta(P) = H_R - R_k. \end{aligned} \tag{6}$$

Every complete legal table obeys [6]

$$0 \le \beta \le \alpha \le \beta + k - 1, \qquad W_P(m) = \lfloor m/k \rfloor + 1 + \max(\alpha, (m \bmod k) + \beta). \tag{7}$$

Thus $(k, \gamma, \alpha, \beta)$ is an exact planner summary for a completed table. It is not a sufficient search state for an unfinished table.

For a canonical cap (a, b), the raw historical tests are $H_0 \le a$ and $H_R \le k - 1 + b$. The second raw cap is not b. All peaks must include the full history.

1.5 Exact recursive objective

A plan is a finite arity sequence $\pi = (k_0, \dots, k_{D-1})$, $2 \le k_i \le K$, with one complete legal table P_k per used arity. Set

$$m_0 = n, \quad N_0 = 1, \quad m_{i+1} = W_{P_{k_i}}(m_i) < m_i, \quad N_{i+1} = N_i(2k_i - 1). \tag{8}$$

All calls at a level use this same arity, table and child width. Repeated arities reuse the same table. The empty sequence is allowed. No monotone-arity requirement is imposed.

For positive integer R, define

$$\begin{aligned} \tau(R) &= \left\lfloor 3.075 \log_2(100R) + 0.86 + \tfrac{1}{2} \right\rfloor \\ &= \left\lfloor \frac{\text{bitlength}((100R)^{615}) - 1 + 272}{200} \right\rfloor. \end{aligned} \tag{9}$$

The constants are exact rationals and rounding is $\lfloor x + 1/2 \rfloor$, not ties-to-even. The second line permits exact evaluation without floating-point logarithms.

The repository's chosen PP benchmark is

$$\boxed{\mathcal{T}(\pi, \{P_k\}) = 8 \sum_{i=0}^{D-1} N_i(m_{i+1} - 1)\gamma(P_{k_i}) + R(4 + \tau(R)), \qquad R = N_D m_D^2.} \tag{10}$$

Define $\mathrm{OPT}(n, K)$ as the minimum of (10) over all feasible plans and legal tables. Formula (10) is not KMY's original CR_ϕ-count estimator. Exact optimization of this rounded cost does not mean exact Clifford+T synthesis of arbitrary phase angles.

Output. The algorithm returns the minimum cost, a realizing arity sequence, and one explicit marked instruction table per used arity. Replay must establish all widths, multiplicities, charges and the terminal cost. A feasible table alone does not certify a minimum.

---

2 How the exhaustive algorithm finds tables

2.1 Minimum charge under a width cap

For a canonical pair (a, b), let

$$c_{\min}(k, a, b) = \min\{\gamma(P) : P \text{ complete and legal, } \alpha(P) \le a,\ \beta(P) \le b\}, \tag{11}$$

with $+\infty$ for an empty set. For $\delta \in \{0, \dots, k-1\}$, define

$$\theta(k, \delta, C) = \min\{a \ge \delta : c_{\min}(k, a, a-\delta) \le C\}. \tag{12}$$

Set $\min \varnothing = +\infty$, so an infeasible ray has infinite threshold. A used table has $\alpha \le n-2$: its child width is at least $\alpha + 1$ and at most $n-1$. Therefore only thresholds at most $A_n = n-2$ matter. At $n = 1$, root stop is the only plan.

The oracle returns the exact threshold and a witness when $\theta \le A_n$, or reports $\theta > A_n$. The latter is not a global infinity claim unless the full sufficient cutoff was searched.

2.2 Why the search is finite

The reviewed algorithm uses three established facts [4, 7, 5]. The following statements and proof summaries identify the mathematical dependencies.

Lemma 2.1 (Charge ceiling and incumbent)**.** *Every feasible cap has a minimum-charge witness of charge at most $27k^3$. An explicit paired F.2 constructor has charge $c_0 \le 6k^2$ and profile (a_0, b_0) with $a_0, b_0 \le 8k^2$.*

The paired constructor groups opposite points. It prepares even- and odd-degree coefficient sums in two existing rows, emits the two literal target sums using addition/subtraction, and reverses the preparation. It forms and reverses an unpaired final target directly. Negative-reciprocal signs are included. This construction supplies an incumbent, not a restriction on candidate tables.

For the charge ceiling, the potential $\sum_j u_j - \log_2 |\det X|$ increases by at least one at each charged operation and is unchanged by exact shifts. Identity endpoints therefore imply $\gamma \le 3ka$ under zero-seed cap a. For fixed δ, put $a_* = \max(a_0, b_0 + \delta) < 9k^2$. Below a_*, the potential bounds the minimum by $27k^3$; at and above a_*, the constructor costs at most $6k^2$. The constructor bounds are proved in the retained source package, not inferred from its finite tests.

Lemma 2.2 (Primitive normalization)**.** *Every complete legal table has a replacement preserving signed operation counts and literal marker order, with no greater historical width in either scan. Rows are primitive between charged macros. A minimum witness needs $O(k^3)$ primitive instructions.*

Write $X_i = 2^{g_i} Y_i$, with Y_i primitive, and replace width u_i by $u_i - g_i$. The row-mass invariant $\|X_i\|_1 \le 2^{u_i}$ makes this nonnegative. Absorb isolated shifts into the next addition. A negative relative source exponent becomes a destination left shift. After addition, divide by the row's common power of two. The invariant proves that every transformed transient peak is no greater than the corresponding old peak. Primitive literal targets and identity endpoints are preserved. A conservative length bound is $3A + \nu + q$, for A additions and ν negations, plus four optional padding instructions.

Lemma 2.3 (Threshold saturation)**.** *For $C < c_0$, a finite threshold satisfies*

$$\theta(k, \delta, C) \le S(k), \qquad S(k) = (6k^2+1)(6k^2)!(8k^2)^{6k^2} = 2^{O(k^2 \log k)}. \tag{13}$$

For $C \ge c_0$, it satisfies $\theta(k, \delta, C) \le a_* < 9k^2$.

In the low-budget proof, each scalar coefficient of a normalized operation pattern is a signed sum of powers of two. Finite alternatives for exponent order, small gaps and separated cancellation clusters give integer linear constraints. They enforce integrality at every prefix, all literal markers, identity restoration and both histories. At a primitive boundary, if either relative pre-addition exponent is positive, the other exponent is zero and parity makes the sum primitive. Otherwise both pre-addition exponents are zero and only the normalization-division exponent can vary. Thus a fixed addition template has at most one exponent variable; the cap adds one more. There are at most $6k^2$ variables in this budget range, with coefficient and constant bounds $8k^2$.

The small-integer-point lemma used by the proof is

$$\begin{aligned} & B \in \mathbb{Z}^{s\times t}, \quad d \in \mathbb{Z}^s, \quad t, M \in \mathbb{Z}_{\geq 1}, \\ & \{x \geq 0 : Bx \leq d\} \cap \mathbb{Z}^t \neq \varnothing, \quad |B_{ij}|, |d_i| \leq M \\ \Longrightarrow \quad & \text{some feasible integer } y \text{ has } 0 \leq y_i \leq (t+1)t!M^t. \end{aligned} \tag{14}$$

Determinant bounds on vertices and integer recession rays, followed by removal of integer ray multiples, prove (14), including lower-dimensional systems via homogenization. This gives (13). The high-budget bound uses the explicit constructor. The symbolic argument proves a cutoff; the executable search below does not need to enumerate the symbolic systems.

2.3 Normalized macro enumeration

For each (k, δ, C), set

$$\Lambda = \begin{cases} \min(n-2, S(k)), & C < c_0, \\ \min(n-2, \max(a_0, b_0 + \delta)), & C \geq c_0. \end{cases} \tag{15}$$

If $n - 2 < \delta$, report $\theta > n - 2$. Budget zero is infeasible for $k \geq 2$, because a nonbasis target must be created.

A state contains

$$(Y, u^0, u^R, H_0, H_R, S, c), \tag{16}$$

where Y is the primitive integer matrix, S is the marked-target set and c is weighted charge. Store predecessor information for reconstruction. Initially $Y = I_k$, $c = 0$, and the width seeds are those of (6). Initial targets can be marked immediately.

Enumerate these choices:

1. **Addition.** Choose $d \neq h$, $\varepsilon = \pm 1$ and $p_d, p_h \geq 0$, with $\min(p_d, p_h) = 0$, $p_d, p_h \leq \Lambda - 1$. Form

$$Z = 2^{p_d} Y_d + \varepsilon 2^{p_h} Y_h, \qquad t = \min_{j: Z_j \neq 0} \operatorname{val}_2(|Z_j|).$$

   Before division, test

$$v^s = 1 + \max(u_d^s + p_d, u_h^s + p_h), \qquad s \in \{0, R\},$$

   against raw caps Λ and $k - 1 + \Lambda - \delta$. Record these peaks. If valid, set $Y_d = Z/2^t$, $u_d^s = v^s - t$, and add charge 1 or 3. Expand the macro as a destination left shift when needed, the permitted addition, and an exact right shift when $t > 0$.

2. **Negation.** Negate a selected row, increase its scanner width by one in both scans, check both caps, and add charge one.

3. **Mark and accept.** At each primitive boundary, emit every newly available required literal marker once. Choose a deterministic row if needed. Markers are free and value-neutral, and their order is unrestricted. Accept only states with all targets marked and $Y = I_k$.

Reject any prefix of weighted charge greater than C. Each macro has positive charge, so depth is at most C. Depth is not a substitute for the weighted-budget test.

For an accepted word P, its least cap on this ray is

$$a_{\mathrm{ray}}(P) = \max(\delta, \alpha(P), \beta(P) + \delta). \tag{17}$$

Return the minimum of these values with its explicit word. If no candidate exists, return $\theta > n - 2$. Only exhaustion of the full untruncated sufficient cutoff establishes that no finite threshold exists.

2.4 Construct the minimum-charge catalogue

There are k differences and $27k^3 + 1$ charge budgets. Thresholds determine

$$c_{\min}(k, a, a - \delta) = \min\{C : \theta(k, \delta, C) \le a\}. \tag{18}$$

A witness below cap (a, b) can be given exactly that canonical profile by prepending

$$\mathrm{left}(0, a),\ \mathrm{right}(0, a),\ \mathrm{left}(k - 1, b),\ \mathrm{right}(k - 1, b). \tag{19}$$

This prefix has zero charge. It restores the matrix and current widths, raises only historical peaks, and adds no markers. Hence cap-minimum and exact-profile minimum charges agree.

The catalogue must retain the needed charge–width choices. Keeping only the cheapest table per arity is insufficient. An offset interval also cannot automatically be replaced by its smallest endpoint: optimized continuation cost need not be monotone in width.

3 Why the answer is optimal

3.1 Local optimality

Theorem 3.1 (Threshold and cap-minimum realization)**.** *The finite macro search returns the exact truncated threshold with a legal witness. Equations (18)–(19) give a minimum-charge table at each requested exact canonical profile.*

Proof. Every returned word is sound by exact replay of primitives, markers, caps and identity restoration. Conversely, a genuine witness has a no-wider, same-charge normalized representative by Lemma 2.2. Lemma 2.3 supplies the sufficient finite range whenever a threshold is finite. Every macro of that representative occurs in the search alphabet, and its total length is within the charge-depth bound. Earliest marking loses no word because targets are primitive and marker order is unrestricted. Therefore a minimizing representative is considered. Taking the least ray cap gives the threshold. Taking the least successful budget gives (18). Padding gives the exact requested profile without additional charge. □

This is minimum charge *at a specified profile*. A recursively selected table need not have minimum charge without a width constraint.

3.2 Global recursive optimization

Let root-stop cost be

$$U = n^2(4 + \tau(n^2)). \tag{20}$$

An optimum costs at most U. Since positive-width terminal cost is at least its multiplicity, every competitive plan satisfies

$$Q = N_D = \prod_i (2k_i - 1) \le U, \qquad D \le \lfloor \log_3 U \rfloor. \tag{21}$$

The retained exact outer procedure [5] does the following:

1. Enumerate arity sequences with $Q \le U$, decorated by residues $r_i = m_i \bmod k_i$, profile differences δ_{k_i}, and charges c_{k_i}. Repeated arities must agree on difference and charge.
2. Along a fixed (k, δ), the minimum charge is nonincreasing in the offset a. A fixed charge label occupies one integer interval. Assign one integer offset variable a_k in that interval to each distinct used arity.
3. For each terminal width $z = 1, \dots, n$, enforce exact recursion, strict contraction and the objective by exact integer feasibility. Find the least budget and recover the offsets.
4. Compare every feasible result with root stop. Reconstruct one padded witness per used arity and reuse it everywhere.

The width elimination in step 3 is explicit. Set $P_0 = 1$, $P_{i+1} = P_i k_i$, $h_i = 1 + \max(0, r_i - \delta_{k_i})$, and define affine expressions

$$x_0 = n, \qquad x_{i+1} = x_i + P_{i+1}(a_{k_i} + h_i) - P_i r_i, \qquad m_i = x_i / P_i. \tag{22}$$

Impose $x_D = P_D z$. All later increments are divisible by P_{i+1}, so reduction modulo P_{i+1} enforces the required integer width and residue at every earlier level. Strict contraction is

$$k_i x_i - x_{i+1} \ge P_{i+1}. \tag{23}$$

For fixed z, the leaf cost is constant and the remaining cost is affine in the offsets. Multiplication by P_D clears denominators. Integer budget queries recover the exact minimum.

Theorem 3.2 (Global exactness in the declared model)**.** *Complete local threshold results and exact outer optimization return* $\mathrm{OPT}(n, K)$*, a recursion sequence and a realizing shared-table family.*

Proof. Take any feasible plan of cost at most U. Every used offset satisfies $a \le n - 2$. Replace each arity's table once by a minimum catalogue witness with the *same exact profile*, using padding. Equation (7) preserves all widths, residues, multiplicities and admissibility conditions, while charge does not increase. Thus some catalogue plan is at least as good as any unrestricted competitor. Bounds (21) include it in the outer search. Shared offset variables enforce one table per arity. Conversely, every accepted outer solution reconstructs legal tables and an admissible plan. The two optimum values are equal. □

3.3 Feasibility certificates and optimality certificates

Replaying a table proves its feasibility and charge. Replaying a plan proves its achievable T-count. Neither replay excludes a better answer. An optimality claim also requires complete local searches for the necessary smaller budgets/caps and complete outer optimization, or sound certificates that cover every excluded alternative.

A timeout, a hash or a solver status string alone is not that proof. No general polynomial-size independently checkable optimality certificate is asserted. Search traces and infeasibility certificates can themselves be large.

3.4 Is verifying optimality efficient?

There are three different verification tasks.

Verify a feasible answer. Given explicit tables and a recursion sequence, replay every operation using exact integers. Check target coverage, identity restoration, both histories, repeated-arity consistency, strict contraction and the score (10). This takes polynomial time in the instruction count, arities and integer bit lengths encountered. A successful replay proves an upper bound on the optimum.

For a used table, its zero-seed history is at most $n-2$. The row-mass bound then gives $O(n)$-bit coefficients. The minimum-charge ceiling and primitive normal form provide $O(k^3)$-instruction witnesses per used arity, with shifts encoded using $O(\log(n+1))$ bits after padding. Together with $D = O(\log(n+1))$, this gives polynomial-size feasible witnesses in the measure $n + K + \text{bitlength}(B)$. Thus the budget decision *does some feasible plan have cost at most B?* is in NP when n, K are encoded in unary and B in binary. If $B \geq U$, the empty plan suffices; otherwise a feasible plan has $D \leq \log_3 U$. This is not a binary-n certificate claim.

Verify that no better answer exists. For a feasible incumbent of integer cost B, optimality additionally requires

$$\nexists\, (\pi, \{P_k\}) \text{ feasible with } \mathcal{T}(\pi, \{P_k\}) \leq B - 1.$$

A cheaper plan can be certified by a feasible witness. The absence of every cheaper plan is a different requirement. No uniform polynomial-time verifier with a guaranteed polynomial-size optimality certificate has been established here. The NP upper bound for cheaper-plan existence does not provide such a certificate.

Verify a supplied proof or exhaustive trace. A sound checker may run in time polynomial in the length of the supplied proof. That proof or trace can itself be exponentially long in the problem parameters. Polynomial verification in certificate length therefore does not imply polynomial verification in the original input size. A matching independently proved lower bound can make an individual instance easy to certify, as in the $k = 2$ example.

For the current method, one general way to validate optimality is to complete the exact searches and outer optimization described above. Its available guarantee remains (26): polynomial in numeric n for each fixed K, but with no uniform polynomial guarantee when K grows. The absence of a known efficient optimality certificate is not a proof that efficient certification is impossible, or that the optimization problem is outside P.

3.5 A complete small example

For $k = 2$, the targets are $0, \infty, -1$. Consider

operation	X_0	X_1	charge
start; mark $0, \infty$	$(1, 0)$	$(0, 1)$	0
$X_0 \leftarrow X_0 - X_1$; mark -1	$(1, -1)$	$(0, 1)$	3
$X_0 \leftarrow X_0 + X_1$	$(1, 0)$	$(0, 1)$	4

Raw peaks are 2 and 3, so the canonical profile is $(2, 2)$. At least two additions are needed to create an off-diagonal target and restore identity. If any addition is negative, their total charge is at least $3 + 1 = 4$. If all additions are positive, a negative coefficient requires a negation. Identity restoration requires an even number of negations, since additions have determinant one and shifts have positive determinant. There are then at least two additions and two negations, again costing at least four. Thus the displayed table is charge-optimal, both without a width cap and under cap $(2, 2)$.

Its child width is $\lceil m/2 \rceil + 3$. At $m = 16$, the child width is 11, and its local cost at multiplicity one is 320. This does not establish that this step is part of the best *whole* recursive plan at $n = 16$.

4 Computational complexity and storage

The following are bit-operation bounds in numeric physical n; they are not polynomial-in-$\log n$ claims.

4.1 Local search

The branching count is $O(k^2(\Lambda + 1))$, with depth at most C. Under the cap, coefficients have $O(\Lambda)$ bits by the row-mass bound, and exact replay costs polynomial time in k, Λ. Thus a query takes

$$(k^2(\Lambda + 1))^{O(C)} \operatorname{poly}(k, \Lambda, \log(n + 1)). \tag{24}$$

For $C < c_0$, use $C = O(k^2)$ and $\log \Lambda = O(k^2 \log k)$. For $C \geq c_0$, use $\Lambda = O(k^2)$ and $C = O(k^3)$. The respective parameter factors are $2^{O(k^4 \log k)}$ and $2^{O(k^3 \log k)}$. The uniform local bound is

$$2^{O(k^4 \log k)} \operatorname{poly}(\log(n + 1)). \tag{25}$$

There are $O(k^4)$ threshold queries per arity and $O(K^5)$ through arity K.

4.2 Outer search and full bound

Each arity occurrence has at most $28k^5$ decorations. The retained counting argument gives fewer than $3U^{10}$ decorated sequences. If t distinct arities occur, then $t! \leq Q \leq U$, so $t^{O(t)} = U^{O(1)}$. The outer proof imports Kannan's exact integer-feasibility theorem [3, Theorem 5.4], in $t^{O(t)}$ poly(encoded input length) time. Only the t offsets are integer variables; the widths in (22) are expressions, not extra variables.

With terminal-width choices, budget searches, local queries and reconstruction, the full bound is

$$\boxed{2^{O(K^4 \log K)} \operatorname{poly}(n).} \tag{26}$$

This is fixed-parameter tractability in K, under the numeric-n convention. This bound does not establish uniform polynomial time for growing K. It supplies no lower bound that excludes a faster algorithm. In particular, the proved parameter factor is not $2^{O(K \log K)}$.

If a separate sufficient-cutoff theorem gave $K = O(\sqrt{\log n})$, substituting it into (26) would give

$$2^{O((\log n)^2 \log\log n)}\,\mathrm{poly}(n),$$

a quasipolynomial upper bound, not a polynomial guarantee. By contrast, a hypothetical factor $2^{O(K \log K)}$ under that same cutoff would be $n^{o(1)}$, and would preserve a $\mathrm{poly}(n)$ total bound. Neither that smaller factor nor a sufficient $\sqrt{\log n}$ cutoff is assumed in this document.

4.3 Time and memory interpretation

The factor U^{10}, hidden constants and local branching can be large. Equation (26) gives no execution time at $K = 11$–15 or $n = 50{,}000$. The ceiling $27k^3 = 91{,}125$ at $k = 15$ is a sufficient search bound, not a prediction of optimal charge.

One matrix can require $O(k^2(\Lambda + 1))$ bits, in addition to scanners, markers and reconstruction data. Cache size depends on the implementation and can be much larger than one state. The time theorem is not a practical RAM estimate; it does not require all possible states to be stored at once.

The outer polynomial bound uses the stated integer-feasibility dependence. A different solver can remain exact without inheriting that bound. The local macro search itself needs no integer-programming solver.

5 Twelve limitations in three variants

Each item gives the same restriction in ordinary, controlled and formal language. The controlled panels follow ASD-STE100 Issue 9 writing principles [2], using the domain nouns defined here. They are descriptive text. Their sentences have at most 20 words, stricter than the descriptive 25-word limit. Complete conformity also depends on dictionary meaning and part of speech. No independent STE conformity certificate is claimed. Equations and formal panels are outside this controlled layer.

5.1 L1. Global optimality has a fixed search domain

Plain language. The optimum covers all legal tables in Section 1. The algorithm does not choose other evaluation points or search all quantum arithmetic circuits. Arbitrary legal helpers and marker order are allowed within the declared operation language.

ASD-STE100 controlled English. The algorithm uses one fixed set of evaluation points. The permitted row operations define the search domain. The global optimum applies only to this search domain. The algorithm does not select a different evaluation-point family.

Formal statement. Let $\mathcal{P}_k$ be exactly the tables accepted by (1)–(4). The minimum is over $\prod_{k\ \mathrm{used}} \mathcal{P}_k$ and admissible plans. No equality is asserted with an optimum over other point sets, extra registers, other primitives or arbitrary quantum circuits.

5.2 L2. The recursion model is restricted

Plain language. All children share a width, and each arity has one table. Every recursive step must reduce width. The guarantee does not extend to independently optimized branches with different widths or tables.

ASD-STE100 controlled English. Each recursion step uses one child width. The child width must be smaller than the parent width. Each arity uses the same table at all recursion steps. Other recursion models are outside the search domain.

Formal statement. Every feasible plan satisfies (8), with one P_k per arity. It is a homogeneous-level recursion sequence. Branch-dependent tables $P_{k,i,\ell}$ and widths $m_{i+1,\ell}$ are outside the model. KMY's broader branch-dependent estimator is not identified with this planner [1, App. F.1].

5.3 L3. The objective is one specified T-count

Plain language. The optimizer minimizes the exact value of a rounded T-count formula. It does not also minimize qubits, depth, physical runtime or error-correction cost. Exact optimization of this formula is not exact synthesis of arbitrary rotation angles.

ASD-STE100 controlled English. The objective is the recursive T-count. The objective does not include qubit count, circuit depth, or hardware time. A global optimum for T-count can have a higher cost for another objective.

Formal statement. The objective is precisely (10), using (9). In general, $\arg\min \mathcal{T}$ is not contained in $\arg\min G$ for another resource G. Charges $1, 3, 1$, free shifts and the leaf cost are fixed model data. The same recursive objective has not been established here for PhaseTripleProduct.

5.4 L4. The arity cutoff limits the guarantee

Plain language. The result is optimal among arities up to K. A larger arity can improve the answer. Practical observations of small arities do not prove that a small cutoff preserves every optimum. The search does not require decreasing arities.

ASD-STE100 controlled English. The input K gives the largest permitted arity. The global optimum can change when K increases. Tests at small arities do not give a proof that larger arities are unnecessary. The search permits arities in any order.

Formal statement. For $K' \geq K$, $\mathrm{OPT}(n, K') \leq \mathrm{OPT}(n, K)$. Equality needs a sufficient-cutoff proof or exact comparison. Neither $K \leq 16$ nor $k_{i+1} \leq k_i$ is assumed. The observed practical range 11–15 is not an admissibility theorem.

5.5 L5. The time bound does not establish practicality

Plain language. The exhaustive search can take extremely long. Hidden constants and the growing arity factor prevent a reliable completion estimate from the asymptotic bound. The polynomial is in operand bitwidth, not the number of bits used to encode that width.

ASD-STE100 controlled English. The search can take a long time. The complexity bound does not give a completion time for a specific computer. The bound uses the operand bitwidth n. It does not use the bit length of n.

Formal statement. Equation (26) is an asymptotic upper bound in bit operations, not a measured machine-time model. If $b_n = \mathrm{bitlength}(n)$, then $\mathrm{poly}(n)$ can be $2^{O(b_n)}$. It does not establish uniform $\mathrm{poly}(n, K)$ time, and it does not establish non-P.

5.6 L6. The outer planner can remain costly

Plain language. Table synthesis is only the local part. The planner must still choose a recursion sequence, stopping level and consistent tables. Its polynomial guarantee allows a large exponent and expensive integer-feasibility calculations.

ASD-STE100 controlled English. The table search is only one part of the algorithm. The planner must also find a recursion sequence and consistent table choices. A polynomial bound can have a large exponent. The planner can therefore need much time.

Formal statement. The outer reduction permits fewer than $3U^{10}$ decorated sequences, n terminal widths and integer-budget feasibility queries. Its dimension is t, the number of distinct arities, with $t! \leq U$. The imported $t^{O(t)}$ algorithm is a theoretical component, not a local benchmark. Another solver requires separate runtime accounting even when it remains exact.

5.7 L7. Minimum local charge is not the recursive objective

Plain language. A table with a higher charge can make its children narrower and reduce total recursive cost. Keeping only the cheapest table per arity can lose the global optimum. The required charge–width alternatives must remain available, with one consistent table per arity.

ASD-STE100 controlled English. A table with a higher charge can give a smaller child width. The smaller width can reduce the recursive T-count. The planner must keep the required charge and width choices. The planner must use the same table when it uses the same arity again.

Formal statement. Minimizing $\gamma(P_k)$ alone need not minimize (10). Safe catalogue replacement preserves the same exact profile (a_k, b_k). A narrower replacement can change later residues and continuation costs. The shared-table rule applies to the whole profile and its realizing witness, not merely to a charge label.

5.8 L8. Sufficient finite bounds can be large

Plain language. The proof supplies finite limits, but they can greatly exceed the values in useful tables. A smaller limit needs a proof that it cannot remove a required optimum. The incumbent construction's charge is not a safe ceiling for every narrower-width query.

ASD-STE100 controlled English. The proof gives finite search limits. These limits can be much larger than the values in a useful table. A smaller limit can remove a table that gives the global optimum. Such a limit needs a separate proof.

Formal statement. The sufficient budget range is $0 \le C \le 27k^3$, with cutoff (15). At $k = 15$, the charge ceiling is 91,125. The large $S(k)$ is used below c_0; the constructor bounds thresholds at budgets at least c_0. Restricting all searches to $C \le c_0$ can lose higher-charge, lower-width alternatives. These bounds do not predict typical optima.

5.9 L9. Exact integers and memory constrain implementation

Plain language. Row coefficients can exceed machine-integer ranges. Overflow, approximate division or approximate equality can change feasibility. Large state caches can exhaust memory. The theorem gives no practical RAM estimate.

ASD-STE100 controlled English. The row coefficients require exact integer arithmetic. Fixed-width integers can overflow. An overflow can change a feasibility result. A state cache can require more memory than the computer has.

Formal statement. Coefficients under cap Λ have $O(\Lambda)$ bits; one matrix can require $O(k^2(\Lambda+1))$ bits. No fixed machine word suffices uniformly. Division, valuation, equality and objective comparison must be exact. Formula (9) removes the real-logarithm oracle. A cache can contain many states, so its space is not bounded by one state's encoding length.

5.10 L10. Faithful semantics are necessary

Plain language. Small implementation changes can alter the problem. Negative additions cost three. Both width histories must pass, including peaks before division. Literal target signs and indexed identity restoration must remain exact.

ASD-STE100 controlled English. The implementation must follow the specified operations and costs. It must do a check of both width histories before each normalization division. Each right shift must be exact. Each marker must match its target row. The final row matrix must be the identity matrix.

Formal statement. The transition relation is exactly (3)–(4), with raw caps a and $k-1+b$. A normalized addition tests $1+\max(u_d+p_d, u_h+p_h)$ before subtracting its valuation. Final I_k and exact-once literal target coverage are required. Final-width checks alone are insufficient. Accepting $\pm v_k(w_\ell)$ interchangeably changes the domain.

5.11 L11. A partial search does not certify a minimum

Plain language. A legal table establishes an achievable cost, not the absence of a better answer. A timeout leaves optimality open unless another complete certificate is available. Failure below a smaller cap does not establish global infeasibility.

ASD-STE100 controlled English. A valid table gives an upper bound for the minimum cost. A timeout does not give a proof of optimality. A failed search below a smaller width limit does not show global infeasibility. The result must identify an incomplete search.

Formal statement. An incumbent of cost B proves only $\mathrm{OPT}(n, K) \leq B$. Optimality also requires a valid lower bound $L \geq B$, supplied here by complete exact search. A failed truncated query proves $\theta(k, \delta, C) > n - 2$, not necessarily $+\infty$. A hash or feasible-word replay does not certify the absence of better words.

5.12 L12. The full implementation is not certified

Plain language. The results are reviewed mathematical proofs with component tests. The full optimizer is not machine formalized and was not run at large arity in the reviewed package. The Fortran translation needs its own validation and accurate completion-status reporting.

ASD-STE100 controlled English. The general proofs have independent mathematical reviews. The full algorithm has no complete machine-checked proof. The tests do not cover the full search domain. The Fortran implementation needs its own validation.

Formal statement. Executable correctness requires faithful implementation of the transition relation, bounds, catalogue and exact outer solver. Elementary formal lemmas and bounded checks do not prove the whole program. They supply neither a full-domain runtime benchmark nor a general compact optimality certificate.

6 Implementation result records

This section specifies how to interpret results from the current algorithm. It does not prescribe a new search strategy.

Status	Required meaning
Feasible incumbent	A replayed table or plan gives the stated achievable cost. No minimum claim follows.
Exact threshold	All necessary candidates for the query were covered; the threshold and witness are exact.
Infeasible within cap	No accepted word exists within the stated cap and budget. The scope must be recorded.
Globally infeasible threshold	The full sufficient saturation range was exhausted without a witness.
Certified recursive optimum	Complete local information and exact outer optimization establish (10)'s minimum for the declared n, K model.
Incomplete search	A time, memory, arithmetic-range or user cutoff prevented completion. A valid incumbent can still be reported.

A reproducible record identifies n, K, literal F.2 targets and signs, charge weights, scanner rules, leaf formula and all cutoffs. A local query also records k, δ, C, Λ, whether its cap was truncated, its witness and completion status.

Practicality records should separate table-search time from outer planner time. They should include visited prefixes or states, maximum coefficient bit length, peak memory, exact-integer library, compiler and build settings. These measurements describe an implementation; they do not change its optimality domain.

6.1 Controlled-English technical vocabulary

The following technical nouns have one fixed meaning in the controlled panels. Row, coefficient, matrix, integer, marker and width history are defined in Section 1.

Arity
Number k of chunks in one recursive division.

Charge
Weighted operation count γ.

Child width
Common integer width $W_P(m)$.

Complexity bound
Proved asymptotic bound on computation.

Evaluation-point family
Fixed F.2 point sets indexed by arity.

Feasibility
Satisfaction of the acceptance conditions.

Global optimum
Minimum of (10) within its declared domain.

Machine-checked proof
Proof accepted by a formal proof checker.

Normalization division
Division by a row's maximal common power of two.

Operand bitwidth
Number n or m of operand bits.

Planner
Part that selects a recursive plan and shared tables.

Recursive T-count
Cost function (10).

Search domain
All permitted tables and recursive plans.

State cache
Search states stored by an implementation.

Table
Complete instruction sequence defined in Section 1.3.

Upper bound
Value at least as large as the quantity being bounded.

Validation
Checks that an implementation satisfies its specification.

References

[1] G. D. Kahanamoku-Meyer and N. Y. Yao. *Fast quantum integer multiplication with zero ancillas*. arXiv:2403.18006v4, 14 November 2024. Sections II–III; Appendix B, equations (24)–(27); Appendices F.1–F.2 and G. https://arxiv.org/html/2403.18006v4. This source supplies notation, PhaseProduct and point-selection context, not the local exhaustive-optimizer theorem.

[2] ASD Simplified Technical English Maintenance Group. *ASD-STE100 Simplified Technical English*, Issue 9, 15 January 2025. Writing rules and dictionary. https://www.asd-ste100.org/STE_faq.html. Controlled panels use short sentences and consistent technical nouns; independent conformity certification is not asserted.

[3] Ravi Kannan. *Minkowski's Convex Body Theorem and Integer Programming*. Mathematics of Operations Research 12(3), 415–440, 1987. Theorem 5.4, with the intermediate-integer bounds included in the bit-complexity accounting. https://doi.org/10.1287/moor.12.3.415.

[4] Local research package, 15 October 2026. *Constructive thresholds: exact FPT algorithm; uniform P remains open*. These establish the finite local enumeration and cutoff.

[5] Local research package, 15 October 2026. *Gap-closure result: polynomial outer optimization*. Its earlier open local-realization obligation is supplied by reference [4].

[6] Local research package, 15 October 2026. *Exact three-integer planner profiles*. Reference [5] replaces the earlier outer-runtime bound of that note.

[7] Local research package, 15 October 2026. *Whole-table primitive normalization*. The scanner-preserving replacement is incorporated into reference [4].

Local sources. The local research packages cited above are internal working notes and are not part of this repository. This document does not claim uniform polynomial time for growing K.